

# Algorithmique & Programmation

## Introduction à la programmation pour le TAL

Eric Jordan

10 octobre 2025

- 1 Les strings
- 2 Opérations sur les strings
- 3 Fonctions et méthodes
- 4 A l'action

# Le type pour stocker du texte

## str — le type des chaînes de caractères

- ▶ Appelé **string** (ou *chaîne de caractères*)
- ▶ Une **séquence** : les caractères sont ordonnés
- ▶ **Immuable** : on ne peut pas la modifier après création
- ▶ Compatible **Unicode** : supporte les accents, les emojis et les caractères de toute langue du monde

# Créer une chaîne de caractères

Plusieurs façons de créer un string :

- 1 Avec des apostrophes : 'Une chaîne'
- 2 Avec des guillemets : "Une autre"
- 3 Avec des triples quotes :

```
ceci = '''Comme ceci'''  
cela = """Ou comme cela"""
```

Utile pour les textes multi-lignes ou la documentation (docstrings).

# Pourquoi utiliser le triple ?

Pour écrire des chaînes de plusieurs lignes :

```
texte = """Vous savez, moi je ne crois pas'  
qu'il y ait de bonne ou de mauvaise situation.  
Moi, si je devais résumer ma vie 'aujourd'hui...  
"""
```

**A noter :** Python conserve les sauts de ligne et les espaces.

# Convertir d'autres données en string

## Fonction `str()`

Permet de transformer (presque) n'importe quel objet en chaîne de caractères.

```
nombre = 42
texte = str(nombre)
print(texte)           # "42"

reel = 4.2
print(str(reel))       # "4.2"

booleen = True
print(str(booleen))    # "True"
```

# La chaîne vide

Il est possible de créer une chaîne sans aucun caractère : **la chaîne vide**.

```
chaine_vide = ""
```

## Pourquoi faire ?

- ▶ Pour construire une chaîne progressivement

```
bouteilles = 99
base = ''
base += "stock actuel : "
base += str(bouteilles)
print(base) # stock actuel : 99
```

- ▶ Pour tester si une chaîne est vide :

```
if chaine == "":
    print("Vide !")
```

# Échapper des caractères spéciaux

Certains caractères ont une signification spéciale dans une chaîne. Si on en a besoin, on les **échappe** avec une barre oblique inverse :

\

```
texte = 'C\'est un bel après-midi'
print(texte)  # C'est un bel après-midi

chemin = "C:\\Users\\Alice\\Documents"
print(chemin) # C:\Users\Alice\Documents

print("Ligne 1\nLigne 2")  # \n crée un saut de ligne
print("Tab\tSéparé")       # \t insère une tabulation
```



# Concaténation (+) et duplication (\*)

## Concaténation :

```
mot = "bon" + "jour"
```

assembler plusieurs chaînes avec +

```
print(mot)    # bonjour
```

## Duplication : répéter une chaîne avec \*

```
rire = "ha" * 3
```

répéter une chaîne avec \*

```
print(rire)   # hahaha
```

# Opérateur [] : accès aux caractères

On peut accéder à un caractère précis d'une chaîne grâce à son **indice** :

```
mot = "Python"  
print(mot[0])  
print(mot[1])  
print(mot[5])  
print(mot[6])
```

Les indices commencent à 0 :

P(0) y(1) t(2) h(3) o(4) n(5)

On peut aussi accéder depuis la fin avec des indices négatifs :

```
print(mot[-1])    # n  
print(mot[-2])    # o
```

# Opérateur [::] : slicing

Le slicing permet d'extraire une **sous-chaîne** :

`chaîne[début:fin:pas]`

- ▶ `début` : l'indice de départ (inclus, optionnel- 0 par défaut)
- ▶ `fin` : l'indice d'arrêt (exclu, optionnel- fin du mot par défaut)
- ▶ `pas` : intervalle entre caractères (optionnel, 1 par défaut) - utilisé pour «sauter» des caractères ou changer le «sens» de parcours

```
alphabet = "abcdefghijklmnopqrstuvwxyz"
```

```
print(alphabet[0:5])    # abcde
print(alphabet[:5])     # abcde (depuis le début)
print(alphabet[20:])    # uvwxyz (jusqu'à la fin)
print(alphabet[::2])    # acegikmoqsuw
print(alphabet[::-1])   # zyxwvutsrqponmlkjihgfedcba
```

# Opérateur : in

Permet de tester si une **sous-chaîne** est contenue dans une autre.

```
mot = "programmation"

print("gram" in mot)      # True
print("z" in mot)        # False
```

On peut aussi tester le contraire avec `not in`:

```
if "java" not in mot:
    print("Ce n'est pas du Java !")
```

# Longueur d'une chaîne

Pour connaître le nombre de caractères dans une chaîne, on utilise la fonction native `len()` :

```
print(len("bonjour"))    # 7
```

# Les méthodes de chaînes

Avec les strings, on dispose aussi de nombreuses **méthodes**, c'est-à-dire des fonctions associées à l'objet lui-même :

```
texte = "bonjour à TOUS"  
texte.upper()    # "BONJOUR À TOUS"  
texte.lower()    # "bonjour à tous"  
texte.capitalize() # "Bonjour à tous"  
texte.replace("bonjour", "salut") # "salut à tous"
```

**A noter :** on appelle une méthode avec le point : `objet.méthode()`

# split()

La méthode `split()` découpe une chaîne en une liste de morceaux.

```
phrase = "un deux trois"  
phrase.split() # ["un", "deux", "trois"]
```

```
csv = "Jean,Paul,Marie"  
csv.split(",") # ["Jean", "Paul", "Marie"]
```

**Astuce :** par défaut, `split()` sépare selon les espaces.

# join()

La méthode `join()` fait l'inverse de `split()` : elle **assemble** les éléments d'une liste en une seule chaîne.

```
mots = ["un", "deux", "trois"]  
" ".join(mots) # "un deux trois"
```

```
"-".join(mots) # "un-deux-trois"
```

**Astuce :** le texte avant le `.join()` indique le séparateur entre les éléments.



## find() et index()

Ces deux méthodes permettent de **chercher une sous-chaîne** dans une chaîne.

```
texte = "programmation"
```

```
texte.find("gram")    # "un deux trois"
```

```
texte.index("gram")   # "un deux trois"
```

```
texte.find("z")
```

```
texte.index("z")
```

*Que se passe-t-il pour la dernière ligne ?*

### Différence clé

- ▶ `find()` renvoie `-1` si la sous-chaîne n'est pas trouvée.
- ▶ `index()` lève une `ValueError` dans ce cas.

# Résumé des méthodes

| Méthode                   | Définition                                                                 | Exemple (résultat)                                      |
|---------------------------|----------------------------------------------------------------------------|---------------------------------------------------------|
| <code>split()</code>      | Sépare la chaîne selon un séparateur (défaut espace) et renvoie une liste. | <code>"a,b,c".split(',') ⇒ ['a', 'b', 'c']</code>       |
| <code>join()</code>       | Concatène les éléments d'une liste avec un séparateur.                     | <code>',' .join(['a','b','c']) ⇒ 'a,b,c'</code>         |
| <code>find()</code>       | Indice de la première occurrence.                                          | <code>"hello".find('l') ⇒ 2</code>                      |
| <code>count()</code>      | Nombre d'occurrences d'une sous-chaîne.                                    | <code>"banana".count('a') ⇒ 3</code>                    |
| <code>lower()</code>      | Minuscules.                                                                | <code>"Hello".lower() ⇒ 'hello'</code>                  |
| <code>upper()</code>      | Majuscules.                                                                | <code>"Hello".upper() ⇒ 'HELLO'</code>                  |
| <code>title()</code>      | Majuscule à chaque mot.                                                    | <code>"hello world".title() ⇒ 'Hello World'</code>      |
| <code>capitalize()</code> | Majuscule à la 1ère lettre, le reste en minuscules.                        | <code>"hello WORLD".capitalize() ⇒ 'Hello world'</code> |
| <code>swapcase()</code>   | Inverse la casse.                                                          | <code>"Hello".swapcase() ⇒ 'hELLO'</code>               |
| <code>strip()</code>      | Supprime espaces aux bords.                                                | <code>" test ".strip() ⇒ 'test'</code>                  |
| <code>replace()</code>    | Remplace une sous-chaîne.                                                  | <code>"chat".replace('ch','r') ⇒ 'rat'</code>           |
| <code>index()</code>      | Indice (erreur si absente).                                                | <code>"hello".index('l') ⇒ 2</code>                     |

# Formattage de chaînes

Il existe plusieurs façons d'insérer des valeurs dans une chaîne.

## 1. Les f-strings (recommandées depuis Python 3.6)

```
prenom = "alice"  
age = 25  
f"{nom} a {age} ans"  
f"{nom.capitalize()} aura {age + 1} ans bientôt"
```

## 2. L'ancienne façon : opérateur %

```
"%s a %d ans" % (prenom, age)
```

## 3. La méthode format()

```
"{0} a {1} ans".format(nom, 25)  
"{nom} a {age} ans".format(nom="Alice", age=25)
```

# Exercices

- 1 On a 7 prénoms : Jack, Kack, Lack, Mack, Nack, Oack, Pack et Qack. Ecrivez un petit script qui génère tous ces noms à partir des deux chaînes suivantes : prefixes = 'JKLMNOPQ' et suffixe = 'ack'
- 2 Ecrivez un script qui détermine si une chaîne contient ou non le caractère «e»
- 3 Ecrivez un script qui parcourt une chaîne : lorsqu'il rencontre une voyelle, il l'affiche, si c'est une consonne, il met juste «\*»
- 4 Ecrivez un script qui recopie une chaîne (dans une nouvelle variable), en insérant des astérisques entre les caractères
- 5 Ecrivez un script qui recopie une chaîne (dans une nouvelle variable) en l'inversant

## Exercices 2 – Fonctions et méthodes de chaînes I

- ❶ Demandez à l'utilisateur de saisir une phrase et affichez le nombre de caractères qu'elle contient.
- ❷ À partir de la chaîne `"Python est génial"`, affichez :
  - ▶ le texte en majuscules ;
  - ▶ le texte en minuscules ;
  - ▶ le texte avec une majuscule au début.
- ❸ Modifiez la chaîne `"bonjour tout le monde"` pour qu'elle devienne `"salut tout le monde"`.
- ❹ À partir de la phrase `"le ciel est bleu"`, découpez la phrase en une liste de mots puis affichez le deuxième mot.
- ❺ À partir de la liste `["le", "ciel", "est", "bleu"]`, assemblez-la :
  - ▶ avec des espaces ;
  - ▶ puis avec des tirets.

## Exercices 2 – Fonctions et méthodes de chaînes II

- 6 Avec `texte = "programmation python"` :
  - ▶ trouvez la position du mot `"python"` ;
  - ▶ que renvoie `find("java")` ?
  - ▶ que se passe-t-il avec `index("java")` ?
- 7 Découpez la chaîne `"un,deux,trois,quatre"` en une liste, puis réassemblez-la avec des espaces pour obtenir `"un deux trois quatre"`.
- 8 Avec `prenom = "alice"` et `age = 30`, affichez :
  - ▶ `"Alice a 30 ans"` ;
  - ▶ `"L'année dernière, Alice avait 29 ans"` ;
  - ▶ `"ALICE a 30 ans"`.

**Wordle** (Une description complète se trouve ici)

Le joueur doit deviner un mot de 5 lettres en 6 essais. À chaque essai, le jeu indique pour chaque lettre :

- ▶  si elle est correcte et bien placée
- ▶  La lettre en minuscule si elle est dans le mot, mais mal placée
- ▶  si elle n'apparaît pas dans le mot

Par exemple, avec mot cible arbre :

|         |                                                                                   |                                                                                   |                                                                                   |                                                                                   |                                                                                   |
|---------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| poule → |  |  |  |  |  |
| carte → |  |  |  |  |  |
| arbre → |  |  |  |  |  |